Lecture Notes:
Feature Scaling

CIC1205 - Machine Learning

Prof. Eduardo Bezerra

April 6, 2026

0 Contents

1 Introduction **3**
1.1 Motivation . . . 3
1.2 Why Feature Scaling Matters . . . 3
1.3 Overview of Scaling Methods . . . 5

2 Min-Max Scaling (Normalisation) **5**
2.1 Concept . . . 5
2.2 Properties . . . 6
2.3 When to Use Min-Max Scaling . . . 6
2.4 Implementation . . . 7

3 Standardisation (Z-score Normalisation) **7**
3.1 Concept . . . 7
3.2 Properties . . . 7
3.3 When to Use Standardisation . . . 8
3.4 Implementation . . . 8

4 Robust Scaling **8**
4.1 Concept . . . 8
4.2 Properties . . . 9
4.3 When to Use Robust Scaling . . . 9
4.4 Implementation . . . 9

5 Log Transformation **10**
5.1 Concept . . . 10
5.2 Properties . . . 11
5.3 When to Use Log Transformation . . . 11
5.4 Implementation . . . 12

6 Best Practices **12**
6.1 Scaling as Part of the Pipeline . . . 12
6.2 Fit Only on Training Data . . . 14
6.3 Choose the Right Scaler Based on Data Distribution . . . 14
6.4 Apply the Same Scaler to Training and Test Sets . . . 15
6.5 Know When Scaling Is Unnecessary . . . 16
6.6 Storing the Fitted Scaler for Production . . . 16
6.7 Common Mistakes . . . 16

7 Feature Scaling in the Model Selection Pipeline **17**
7.1 Where Does Scaling Fit? . . . 17
7.2 Correct Workflow . . . 17
7.3 Scaling as a Hyperparameter . . . 18
7.4 Implementation with Pipelines . . . 18

8 Summary and Practical Guidance **19**
8.1 Comparative Overview . . . 19
8.2 Decision Guide . . . 19

8.3 The Pipeline Principle 20

1 Introduction

1.1 Motivation

Most machine learning algorithms operate on numerical inputs, but those inputs can differ dramatically in their scales. A dataset might include a feature such as `age` measured in years alongside a feature such as `income` measured in thousands of dollars. Even though both features are numerical, their ranges are fundamentally different, and many algorithms are sensitive to such differences. **Feature scaling** is the process of transforming numerical features so that they fall within a comparable range or share a particular statistical property.

Feature Scaling

Feature scaling is the process of transforming the values of numerical features so that they fall within a certain range or have a particular statistical property (such as mean 0 and standard deviation 1). The goal is to make features comparable, especially when they have different units or scales, and to improve the performance of certain machine learning algorithms.

1.2 Why Feature Scaling Matters

There are three main reasons to apply feature scaling before training a machine learning model.

Equal weighting of features. Some learning algorithms rely on distance computations, such as k-nearest neighbours (k-NN), support vector machines (SVM), and k-means clustering. These algorithms will give disproportionate weight to features with larger numeric ranges. For example, if `income` ranges from 0 to 100 000 and `age` ranges from 0 to 100, a nearest-neighbour search will be dominated almost entirely by differences in income. Scaling ensures that all features contribute equally to distance calculations.

Concrete example: k-NN without scaling

Suppose we want to use k-NN with two features: `age` (between 0 and 100) and `income` (between 0 and 100 000). Consider two individuals:

$$A = (30,\ 50{,}000), \qquad B = (40,\ 55{,}000).$$

The Euclidean distance between them is:

$$\sqrt{(30-40)^2 + (50{,}000 - 55{,}000)^2} = \sqrt{100 + 25{,}000{,}000} \approx 5000.$$

The age difference of 10 years contributes only 100 out of 25,000,100, less than 0.001% of the total. In practice, the model **effectively ignores age** and makes its decisions based almost entirely on income. Without scaling, a feature that happens to have a larger numeric range will dominate all distance calculations, regardless of its actual relevance to the prediction task.

Improves model convergence. Some learning algorithms rely on gradient-based optimisation, such as logistic regression, neural networks, and support vector machines. These algorithms are sensitive to the scale of the data. When features have very different magnitudes, the gradient landscape becomes elongated, and the optimiser must take many small steps to navigate it. Scaling the features produces a more isotropic landscape and allows the optimiser to converge faster. Without scaling, a single large-valued feature can dominate the gradient and slow down or even destabilise training.

Geometric intuition for gradient descent

When features have different scales, the loss surface resembles a narrow, elongated valley: it is much steeper in the direction of the large-scale feature than in the direction of the small-scale one. Gradient descent is forced to take small, cautious steps to avoid overshooting along the steep axis, and as a result converges very slowly.
After scaling, the level curves of the loss become more circular. The gradient then points more directly towards the minimum, and the optimiser can take larger, more efficient steps. This effect is especially pronounced for logistic regression, SVMs, and neural networks trained with gradient descent.

Figure 1 illustrates why scaling often helps gradient-based learning algorithms. When different features have very different numeric scales, the loss surface becomes elongated, which makes gradient descent alternate between directions and progress slowly towards the minimum. After scaling, the level curves become more balanced, so the optimisation trajectory is more direct and convergence is typically faster and more stable.

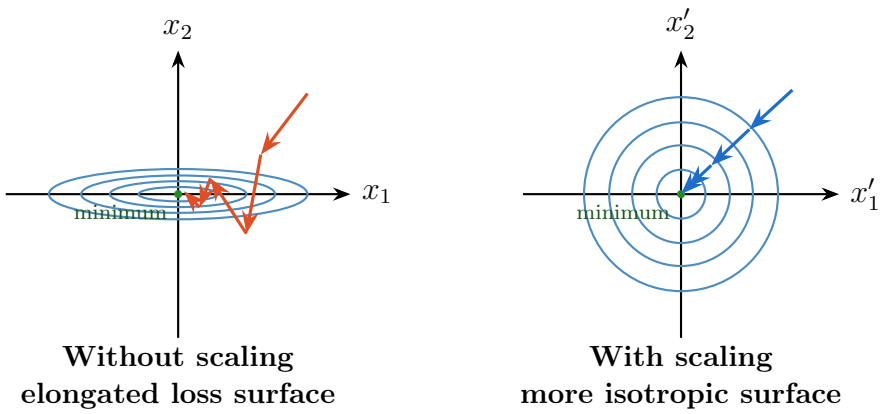

Figure 1: Schematic effect of feature scaling on gradient-based optimisation. On the left, features with very different scales induce an elongated loss surface, causing gradient descent to zig-zag and converge slowly. On the right, after scaling, the loss surface becomes more isotropic, so the optimisation path points more directly towards the minimum.

Improves model accuracy for variance-based methods. Dimensionality reduction techniques such as Principal Component Analysis (PCA) rely on variance and covariance calculations. A feature with a large variance due to its unit of measurement (rather than its informational content) will dominate the principal components. Scaling removes this artificial dominance and allows PCA to identify the directions of true variation in the data.

1.3 Overview of Scaling Methods

These notes cover four main feature scaling techniques. Each method has different properties and is suitable for different data distributions and modelling contexts.

Method	**Formula**	**Best for**	**Outlier sensitivity**
Min-Max Scaling	$x' = \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}$	Features with a fixed range	High
Standardisation	$x' = \dfrac{x - \mu}{\sigma}$	Approximately normal data	Moderate
Robust Scaling	$x' = \dfrac{x - \text{median}}{\text{IQR}}$	Data with many outliers	Low
Log Transformation	$x' = \log(1 + x)$	Right-skewed data	Low

Scaling and model type

Feature scaling is not universally necessary. The table below summarises when scaling is required and when it can be omitted.

Model family	**Scaling required?**
Linear models, SVM, k-NN, Neural Networks, PCA	**Yes**
Tree-based models (Decision Tree, Random Forest, Gradient Boosting)	**No**
Rule-based models	**No**

Tree-based models are invariant to *monotone transformations of individual features*: they split on thresholds, and multiplying a feature by a constant or shifting it by a constant does not change which threshold separates two groups of observations. Scaling therefore has no effect on the structure of a learned tree and can be safely omitted. However, this invariance applies only when trees are used alone. Scaling may still matter when trees are combined with components that *are* scale-sensitive. For example, a pipeline that performs distance-based clustering before fitting a tree, or a stacking ensemble that mixes tree models with a linear meta-learner.
For all model families in the first row of the table, scaling is recommended.

2 Min-Max Scaling (Normalisation)

2.1 Concept

Min-Max Scaling, also called *normalisation*, maps each feature to the interval $[0, 1]$ by subtracting the minimum value and dividing by the range.

Min-Max Scaling

Given a numerical feature X with minimum value $x_{\min}$ and maximum value $x_{\max}$, Min-Max Scaling transforms each observation x as follows:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

The resulting values satisfy $x' \in [0, 1]$. The parameters $x_{\min}$ and $x_{\max}$ are estimated from the training data only.

2.2 Properties

After Min-Max Scaling, the smallest value in the training set maps to 0 and the largest maps to 1. All other values are scaled proportionally. The transformation preserves the shape of the original distribution: if the original data is right-skewed, the scaled data will also be right-skewed.

Sensitivity to outliers. Min-Max Scaling is highly sensitive to extreme values. A single outlier can dramatically compress the remaining values into a narrow sub-interval of $[0, 1]$. For example, if most values of a feature lie between 1 and 10 but one observation has value 1000, after scaling the bulk of the data will be clustered near 0, because $x_{\max} = 1000$ inflates the denominator. When the data contains outliers, Robust Scaling is a safer choice.

Test values can fall outside $[0, 1]$. Because $x_{\min}$ and $x_{\max}$ are estimated from the training set, test observations whose values lie outside the training range will produce scaled values below 0 or above 1 after transformation. This is expected behaviour, not an error. It signals that the test set contains values the scaler has not seen during training, a situation that may warrant further investigation of the data.

Test values outside [0,1]

If a test observation has a value larger than $x_{\max}$ seen during training, its scaled value will exceed 1. Similarly, values below $x_{\min}$ will produce negative scaled values. This is a natural consequence of fitting the scaler only on training data, and it is the correct behaviour. It does not indicate an implementation error. In production, such out-of-range values may indicate data drift and should be monitored.

2.3 When to Use Min-Max Scaling

Min-Max Scaling is appropriate when:

- The feature has a natural, bounded range (e.g., pixel intensities between 0 and 255, or probabilities between 0 and 1).
- The downstream algorithm requires inputs in a fixed range (e.g., some neural network activation functions).
- The data does not contain significant outliers.

2.4 Implementation

In scikit-learn, Min-Max Scaling is implemented by `MinMaxScaler`. As with all scalers, it must be fitted on the training data only and then applied to both the training and test sets.

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)  # fit + transform on
    training data
X_test_scaled  = scaler.transform(X_test)        # transform only on
    test data
```

Custom range

By default, `MinMaxScaler` maps values to $[0, 1]$. A different target range can be specified via the `feature_range` parameter:

```
scaler = MinMaxScaler(feature_range=(-1, 1))
```

This maps the minimum to -1 and the maximum to 1, which is sometimes preferred for neural network inputs.

3 Standardisation (Z-score Normalisation)

3.1 Concept

Standardisation, also called *Z-score normalisation*, transforms each feature to have mean 0 and standard deviation 1 by subtracting the mean and dividing by the standard deviation.

Standardisation

Given a numerical feature X with mean μ and standard deviation σ, Standardisation transforms each observation x as follows:

$$x' = \frac{x - \mu}{\sigma}.$$

The parameters μ and σ are estimated from the training data only.

3.2 Properties

After Standardisation, the training set has mean 0 and standard deviation 1 for each feature (assuming the parameters are estimated on the training set). The transformation centres the distribution around zero and scales it to unit variance. Standardisation does not bound the output to a fixed range; the resulting values can take any real value.

Sensitivity to outliers. Standardisation is less sensitive to outliers than Min-Max Scaling, because it uses the standard deviation rather than the range. However, it is important not to overstate this advantage: both the mean and the standard deviation are affected by extreme values, so their influence on the transformation is reduced but not eliminated. If the data contains severe outliers, Robust Scaling is a safer choice.

Interpretability. The transformed value x' can be interpreted as the number of standard deviations that x lies above or below the mean. This *Z-score* interpretation makes the scaled values more interpretable than those produced by Min-Max Scaling. For example, a standardised value of $+3.0$ indicates an observation that lies three standard deviations above the mean, unusual in a normal distribution, where fewer than 0.3% of observations are expected beyond $\pm 3\sigma$.

3.3 When to Use Standardisation

Standardisation is appropriate when:

- The data approximately follows a normal (Gaussian) distribution, or at least is symmetric and unimodal.
- The downstream algorithm assumes or performs better with zero-mean, unit-variance inputs (e.g., logistic regression, linear SVM, PCA).
- The data does not contain extreme outliers that would substantially distort the estimated mean and standard deviation.

3.4 Implementation

In scikit-learn, Standardisation is implemented by `StandardScaler`.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

The attributes `scaler.mean_` and `scaler.scale_` store the estimated mean and standard deviation for each feature and can be inspected after fitting.

4 Robust Scaling

4.1 Concept

Robust Scaling replaces the mean and range used in Standardisation and Min-Max Scaling with statistics that are less sensitive to extreme values: the median and the Interquartile Range (IQR).

Robust Scaling

Given a numerical feature X with median $\tilde{x}$ and Interquartile Range $\text{IQR} = Q_3 - Q_1$, Robust Scaling transforms each observation x as follows:

$$x' = \frac{x - \tilde{x}}{\text{IQR}}.$$

The parameters $\tilde{x}$, Q_1, and Q_3 are estimated from the training data only.

4.2 Properties

After Robust Scaling, the median of the training set maps to 0, and values one IQR above and below the median map to $+1$ and -1, respectively. Like Standardisation, Robust Scaling does not bound the output to a fixed range: observations far from the median, including outliers, can still produce large absolute values after scaling.

Why median and IQR?

The median depends only on the *ordering* of the data, not on the exact values of individual observations. A single extreme value cannot shift the median as long as it remains in the top or bottom half of the data. The IQR likewise ignores the lowest 25% and highest 25% of observations entirely when computing the spread. This is the key contrast with the mean and standard deviation used in Standardisation: both of those statistics are computed from exact values and are therefore pulled towards extreme observations. The median and IQR are *robust statistics* precisely because they are insensitive to the magnitude of extreme values.

Figure 2 compares the qualitative effect of an outlier on three common scaling methods. Because Min-Max Scaling uses the minimum and maximum values, a single extreme observation can stretch the range and compress the bulk of the data into a narrow interval. Standardisation reduces this effect, but the outlier still influences both the mean and the standard deviation. Robust Scaling is less affected because it relies on the median and the interquartile range, which are much less sensitive to extreme values. This is why Robust Scaling is often preferred when the data contains severe outliers.

4.3 When to Use Robust Scaling

Robust Scaling is appropriate when:

- The data contains outliers or has a heavy-tailed distribution.
- It is important to preserve the structure of the bulk of the data without allowing outliers to distort the scaling parameters.

4.4 Implementation

In scikit-learn, Robust Scaling is implemented by `RobustScaler`.

```
from sklearn.preprocessing import RobustScaler
```

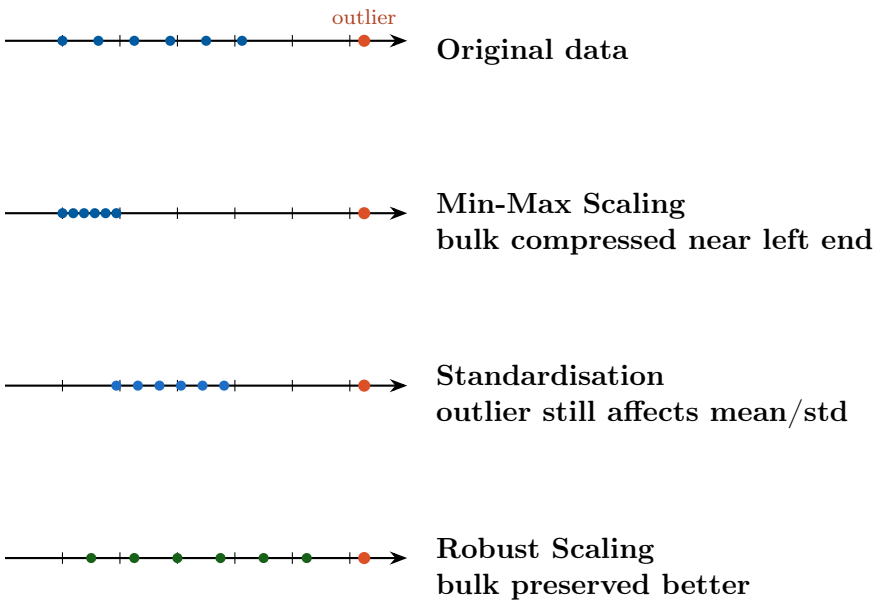

Figure 2: Schematic comparison of how different scaling methods react to an outlier. All four panels use the same horizontal ruler so that the student can compare how the bulk of the data is distributed relative to the outlier. Min-Max Scaling compresses the bulk near one end of the axis; Standardisation reduces but does not eliminate the influence of the outlier; Robust Scaling preserves the structure of the bulk more effectively.

```
scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

5 Log Transformation

5.1 Concept

Log Transformation differs fundamentally from the three methods described in the previous sections. Min-Max Scaling, Standardisation, and Robust Scaling all preserve the shape of the distribution; they only shift and rescale it. Log Transformation, by contrast, is a non-linear operation that actually *changes the shape* of the distribution. It is particularly useful for features with a long right tail, such as income, sales figures, or population sizes.

Scaling vs. transformation: an important distinction

Min-Max Scaling, Standardisation, and Robust Scaling are **linear transformations**: they shift and scale the data without changing the relative spacing between observations. The shape of the distribution is preserved.
Log Transformation is **non-linear**: it compresses large values more than small ones, changing the shape of the distribution. It is therefore not just a scaling step; it is a modelling decision that changes how the feature's information is represented. This distinction matters when interpreting model coefficients and when choosing how to

apply the transformation to new data.

Log Transformation

Given a non-negative numerical feature X, the log transformation is defined as:

$$x' = \log(1 + x).$$

The addition of 1 before taking the logarithm ensures that $x = 0$ maps to $x' = 0$ and avoids the undefined value $\log(0)$.

5.2 Properties

The logarithm is a concave function: it compresses large values more than small ones. Applied to a right-skewed distribution, this compression reduces the long right tail and brings the distribution closer to a symmetric shape. In practice, log transformation can convert a log-normal distribution into an approximately normal one, making subsequent standardisation more meaningful.

Because the transformation changes the shape of the distribution, the original scale of the data is not recoverable without an inverse transformation ($x = e^{x'} - 1$). This inverse is straightforward to compute, but it must be applied whenever a prediction in the original scale is required.

Figure 3 highlights the key conceptual distinction between linear scaling and log transformation. Linear scaling methods may change the range, centre, or spread of a feature, but they preserve the overall shape of its distribution: a right-skewed variable remains right-skewed after such transformations. Log transformation behaves differently. Because it compresses large values more than small ones, it shortens the right tail and can make the distribution substantially more symmetric. This is why log transformation is especially useful for features such as income, sales, or population, whose information is concentrated in multiplicative differences and long positive tails.

Requirement of non-negativity. The formula $\log(1+x)$ requires $x \geq 0$. For features that contain negative values, a shift must be applied first (e.g., $x_{\text{shifted}} = x - x_{\min}$, then $x' = \log(1 + x_{\text{shifted}})$) or a different transformation should be considered.

5.3 When to Use Log Transformation

Log transformation is appropriate when:

- The feature has a strongly right-skewed distribution with a long positive tail.
- The feature takes non-negative values (or can be shifted to be non-negative).
- The relationship between the feature and the target is better captured on a logarithmic scale (e.g., multiplicative rather than additive effects).

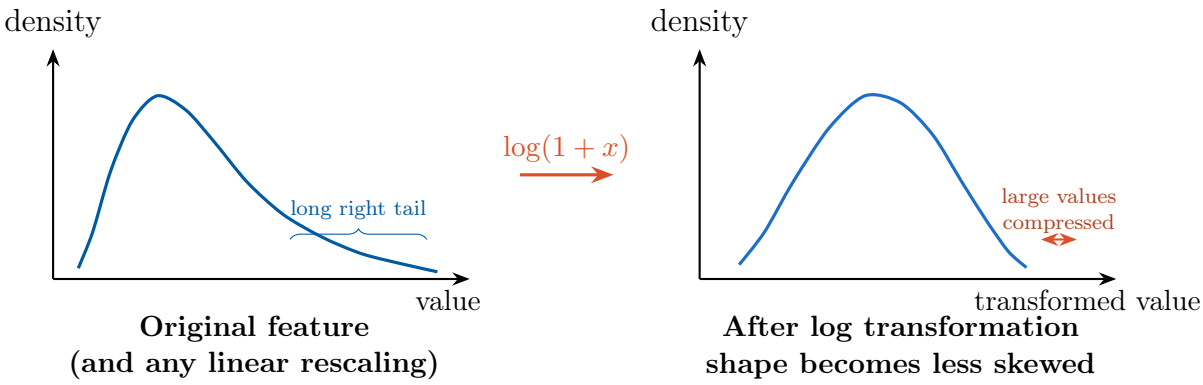

Figure 3: Schematic contrast between linear scaling and log transformation for a right-skewed feature. Linear scaling methods such as Min-Max, Standardisation, and Robust Scaling change location and spread but preserve the overall shape of the distribution, including its long right tail. Log transformation, by contrast, compresses large values more strongly than small ones, reducing skewness and often producing a more symmetric distribution.

5.4 Implementation

Log transformation is typically applied using NumPy's `log1p` function, which computes $\log(1+x)$ in a numerically stable way.

```
import numpy as np

X_train_log = np.log1p(X_train)
X_test_log  = np.log1p(X_test)
```

Log transformation inside pipelines

Unlike scikit-learn scalers, `np.log1p` is a plain function and cannot be used directly inside a `Pipeline`. To integrate log transformation into a pipeline, wrap it in a `FunctionTransformer`:

```
from sklearn.preprocessing import FunctionTransformer
import numpy as np

log_transformer = FunctionTransformer(np.log1p)
```

This object exposes the `fit`/`transform` interface expected by `Pipeline` and `ColumnTransformer`.

6 Best Practices

6.1 Scaling as Part of the Pipeline

Feature scaling is a preprocessing step that must be treated as part of the learning pipeline and evaluated accordingly. The most natural way to enforce this in scikit-learn is to wrap the scaler and the model inside a single `Pipeline` object. This guarantees that:

- `fit` is called only on the training fold during cross-validation;

- `transform` is applied to validation and test folds using the parameters estimated in the fitting step;
- the entire pipeline is evaluated consistently across all cross-validation splits, with no risk of leakage.

Figure 4 summarises the correct role of scaling inside cross-validation. For each fold, the scaler must be fitted only on the training fold; the resulting transformation is then applied to both the training fold and the corresponding validation fold. This prevents leakage, because the validation data does not influence the scaling parameters. The figure also highlights the incorrect alternative of fitting the scaler on the full dataset before splitting.

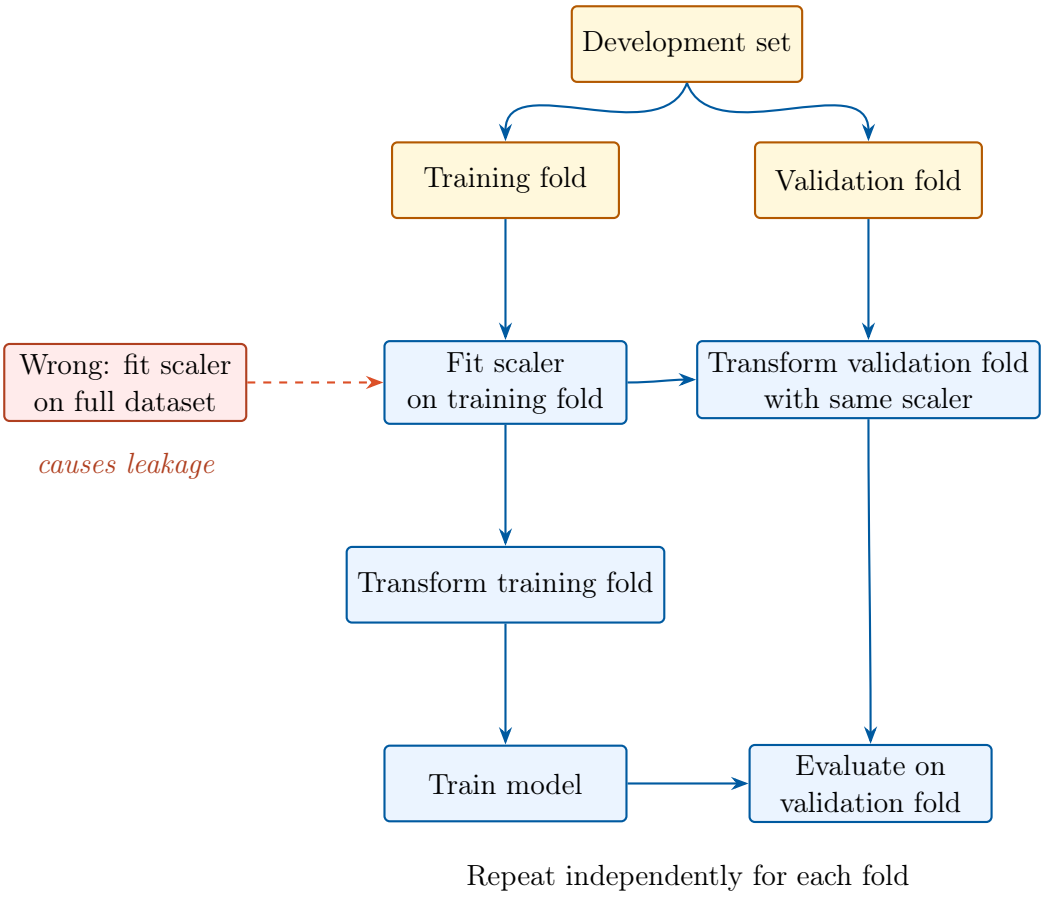

Figure 4: Correct use of feature scaling inside cross-validation. In each fold, the scaler is fitted only on the training fold, then reused to transform both training and validation data. Fitting the scaler on the full development set before the split would leak information from validation data into preprocessing.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('scaler',     StandardScaler()),
    ('classifier', LogisticRegression())
])

pipeline.fit(X_train, y_train)
```

```
y_pred = pipeline.predict(X_test)
```

How a pipeline executes during cross-validation

During each cross-validation fold, the following steps happen automatically:

1. The scaler is fitted on the **training fold only**.
2. The training fold is transformed using the fitted scaler.
3. The **same fitted scaler** is used to transform the validation fold.
4. The model is trained on the scaled training fold.
5. The model is evaluated on the scaled validation fold.

This process is repeated independently for each fold. Because the scaler is re-fitted from scratch in step 1 of each fold, the validation fold statistics never contaminate the scaling parameters.

Pipeline principle for scaling

Always wrap scalers inside a `Pipeline` or `ColumnTransformer`. This ensures that scaling statistics are estimated on training data only, and the same transformation is applied consistently to all other splits.

6.2 Fit Only on Training Data

The most important rule when applying feature scaling is to fit the scaler *exclusively on the training data* and then use the fitted scaler to transform both the training and test sets. Computing scaling statistics (mean, standard deviation, minimum, maximum, median, IQR) on the full dataset before splitting introduces *data leakage*: test-set observations influence the transformation applied to training data, causing the model to "peek" at the test distribution during training and leading to overly optimistic performance estimates.

Data leakage through scaling

The following pattern is **incorrect** and introduces leakage:

```
# WRONG: fitting on the full dataset
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # uses information from test
    set
X_train, X_test = train_test_split(X_scaled, ...)
```

The correct pattern is:

```
# CORRECT: fitting only on training data
X_train, X_test, y_train, y_test = train_test_split(X, y, ...)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

6.3 Choose the Right Scaler Based on Data Distribution

No single scaler is universally best. The appropriate choice depends on the distribution of each feature and on the downstream model. As a practical guide:

- **Min-Max Scaling:** use when features have a natural, bounded range and the data does not contain significant outliers.
- **Standardisation:** use when the data approximately follows a normal distribution and the model assumes zero-mean, unit-variance inputs.
- **Robust Scaling:** use when the data contains outliers that would distort the parameters estimated by Min-Max Scaling or Standardisation.
- **Log Transformation:** use for right-skewed features with a long positive tail.

Different scalers can lead to meaningfully different model behaviour, not only in terms of convergence speed but also in terms of the decision boundaries learned by distance-based and linear models. For this reason, the choice of scaler is best treated as a hyperparameter to be selected by cross-validation (see Section 7).

Visualising the effect of scaling

Before choosing a scaling method, inspect the distribution of each feature with a histogram. After scaling, visualise the transformed distributions to verify that the transformation had the intended effect and did not introduce unexpected distortions.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(X_train_scaled[:, 0], bins=30, kde=True)
plt.title("Feature Distribution After Scaling")
plt.show()
```

6.4 Apply the Same Scaler to Training and Test Sets

The fitted scaler encodes the statistics of the training distribution (e.g., the training mean and standard deviation for `StandardScaler`). It is essential to apply this same fitted scaler, without re-fitting, to the test set so that both sets are transformed in exactly the same way.

When the pipeline includes multiple feature types (i.e., both numerical and categorical), a `ColumnTransformer` can apply the appropriate transformation to each group of columns simultaneously.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

num_features = ['age', 'income', 'height']
cat_features = ['gender', 'city']

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), num_features),
    ("cat", OneHotEncoder(), cat_features)
])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed  = preprocessor.transform(X_test)
```

6.5 Know When Scaling Is Unnecessary

Feature scaling does not always improve model performance. Tree-based models, such as decision trees, random forests, and gradient boosting machines, split on individual feature thresholds and are invariant to monotone transformations of individual features. Scaling has no effect on the structure of the learned tree and can be safely omitted (subject to the caveat about mixed pipelines discussed in Section 1).

When scaling is not required

The following model families are invariant to feature scaling:

- Decision Trees
- Random Forests
- Gradient Boosting (e.g., XGBoost, LightGBM)
- Rule-based models

For these models, spending time on scaling provides no benefit. The effort is better invested in feature engineering or hyperparameter tuning.

6.6 Storing the Fitted Scaler for Production

When a trained model is deployed in production, the same scaler that was fitted on the training data must be applied to all future inputs. To ensure consistency, the fitted scaler should be serialised and saved alongside the model.

```
import joblib

# Save the fitted scaler to disk
joblib.dump(scaler, "scaler.pkl")

# Load and use it in production
scaler = joblib.load("scaler.pkl")
new_data_scaled = scaler.transform(new_data)
```

Failing to save the fitted scaler and instead re-fitting it on production data is a form of distribution shift: production data may have a different mean and standard deviation than training data, and re-fitting would apply a different transformation, silently breaking the consistency between training and deployment.

6.7 Common Mistakes

The following mistakes are frequently encountered in practice and can silently degrade model quality without raising any error.

- **Fitting the scaler before splitting the data.** This is the most common form of data leakage in preprocessing. The scaler must always be fitted on the training set only.
- **Re-fitting the scaler on the test set.** Using a different set of scaling parameters for training and test data breaks the correspondence between the two representations and invalidates evaluation.
- **Forgetting to apply the same transformation in production.** The fitted

scaler must be saved and reloaded alongside the model. A model trained on standardised features will produce incorrect predictions if production inputs are not standardised in the same way.

- **Applying scaling to tree-based models when it is unnecessary.** While harmless in most cases, it adds complexity without benefit.
- **Applying log transformation to features with negative values.** The formula $\log(1 + x)$ is undefined for $x < -1$. A shift must be applied first, or a different transformation chosen.
- **Assuming that standardisation eliminates the influence of outliers.** The mean and standard deviation are both affected by extreme values. Robust Scaling should be used when outliers are a concern.

Entry Ticket 1

Consider a dataset where the feature `income` has values ranging from \$20,000 to \$500,000, with a few individuals earning over \$5,000,000.

1. Which scaling method would be most appropriate for this feature? Justify your answer.
2. A practitioner fits a `MinMaxScaler` on the entire dataset (training and test sets combined) and then splits the data. What problem does this create?
3. After applying `StandardScaler` to the training set, the practitioner observes that a test observation has a scaled `income` value of +8.3. What does this value mean in terms of standard deviations? Is this observation typical or extreme? What might it indicate about the test data?
4. For which of the following models is feature scaling essential, and for which is it unnecessary? (a) Logistic Regression, (b) Random Forest, (c) k-Nearest Neighbours, (d) Gradient Boosting.

7 Feature Scaling in the Model Selection Pipeline

7.1 Where Does Scaling Fit?

Feature scaling is not a one-time preprocessing step performed before model training. It is part of the learning pipeline and must be treated as such. In the model selection context, different scaling methods can produce meaningfully different models, and the best choice must be identified through validation, not by inspecting test performance.

Key Result

Scaling is part of the learning pipeline. Different scaling choices produce different models and must be evaluated within the model selection process.

7.2 Correct Workflow

The correct procedure follows the same logic as hyperparameter selection:

1. Split the data into a development set and a test set.

2. Within the development set:
   - For each candidate pipeline (scaler + model):
     (a) Fit the scaler *only* on the training fold.
     (b) Transform the training fold using the fitted scaler.
     (c) Apply the *same* transformation to the validation fold.
     (d) Train and evaluate the model.
3. Select the best pipeline using validation or cross-validation.
4. Retrain the entire pipeline on the full development set.
5. Evaluate once on the test set.

7.3 Scaling as a Hyperparameter

The choice of scaling method is itself a hyperparameter of the pipeline. To isolate the effect of scaling, one can fix the learning algorithm and vary only the scaling method:

- Min-Max Scaling + Logistic Regression
- Standardisation + Logistic Regression
- Robust Scaling + Logistic Regression

More generally, the search space may combine scaling methods and learning algorithms simultaneously:

- Standardisation + Logistic Regression
- Robust Scaling + SVM
- No scaling + Random Forest

In either case, each combination defines a distinct candidate model and must be evaluated using the same validation strategy. Different scalers can produce different decision boundaries, especially for distance-based and linear models, which is why the choice must be validated rather than decided by intuition alone.

Practical rule

Do not choose the scaling method using the test set. Scaling must be selected using validation or cross-validation, just like any other hyperparameter.

7.4 Implementation with Pipelines

A minimal example combining `Pipeline` and `cross_val_score` is shown below.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipeline = Pipeline([
```

```
    ("scaler", StandardScaler()),
    ("model",  LogisticRegression())
])

# fit() is called only on each training fold;
# transform() is applied to each validation fold automatically
scores = cross_val_score(pipeline, X_dev, y_dev, cv=5)
```

Mental model

Think of the pipeline as a single function:

$$\hat{y} = f(\mathbf{x}_{\text{raw}})$$

where f encapsulates both scaling and learning. Selecting a pipeline means selecting f as a whole; there is no meaningful separation between "choosing a scaler" and "choosing a model."

8 Summary and Practical Guidance

8.1 Comparative Overview

Table 1 summarises the four scaling methods covered in these notes, comparing their effects, best use cases, and sensitivity to outliers.

Table 1: Comparison of feature scaling methods.

Method	**Effect**	**Best use case**	**Outlier sensitivity**
Min-Max Scaling	Scales values to $[0, 1]$	Features with a fixed range (e.g., pixel values)	**High**
Standardisation	Centres at 0, std dev 1	Approximately normal data	Moderate
Robust Scaling	Uses median & IQR	Data with many outliers	**Low**
Log Transformation	Reduces right-skewness	Long-tailed positive data (e.g., income, sales)	**Low**

8.2 Decision Guide

Before scaling any numerical feature, ask yourself these questions:

1. **Does my model require scaling?** If the model is tree-based, scaling is unnecessary and can be skipped.

2. **Does the data contain significant outliers?** If yes, prefer Robust Scaling over Min-Max Scaling or Standardisation.
3. **Is the feature strongly right-skewed?** If yes, consider a log transformation before applying a standard scaler.
4. **Does the feature have a natural bounded range?** If yes, Min-Max Scaling is a natural choice (provided no extreme outliers are present).
5. **Does the downstream model assume approximately normal inputs?** If yes, Standardisation is the most principled choice.

8.3 The Pipeline Principle

In all cases, the scaler must be treated as a learned transformation and fitted only on training data. The correct workflow is always:

1. Fit the scaler on $\mathcal{D}_{\text{train}}$.
2. Apply `transform` to $\mathcal{D}_{\text{train}}$ to produce scaled training features.
3. Apply `transform` to $\mathcal{D}_{\text{val}}$ and $\mathcal{D}_{\text{test}}$ using the parameters estimated in step 1.

This principle extends to cross-validation: the scaler must be re-fitted inside each training fold, not on the full development set. Wrapping scalers in a `Pipeline` object enforces this correctly.

Golden rule of feature scaling

Any scaling that learns parameters from the data must be fitted exclusively on the training portion. Scaling the full dataset before splitting is a form of data leakage.

Meta-rule

If your model depends on **distances or gradients**, scaling is almost always required. If it depends on **thresholds** (tree-based models), scaling is generally irrelevant. When in doubt, include scaling inside the pipeline and let cross-validation decide.

8 References

- scikit-learn preprocessing documentation:
  https://scikit-learn.org/stable/modules/preprocessing.html
- scikit-learn `Pipeline` documentation:
  https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
- scikit-learn `ColumnTransformer` documentation:
  https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html